

Large Language Models Are Universal Recommendation Learners

Junguang Jiang^{1*} Yanwen Huang^{1*} Bin Liu¹ Xiaoyu Kong¹ Ziru Xu¹ Han Zhu¹ Jian Xu¹ Bo Zheng¹

Abstract

In real-world recommender systems, different tasks are typically addressed using supervised learning on task-specific datasets with carefully designed model architectures. We demonstrate that large language models (LLMs) can function as **universal recommendation learners**, capable of handling **multiple tasks within a unified input-output framework**, eliminating the need for specialized model designs. To improve the recommendation performance of LLMs, we introduce a **multimodal fusion module** for item representation and a **sequence-in-set-out approach** for efficient candidate generation. When applied to industrial-scale data, our LLM achieves **competitive results** with expert models elaborately designed for different recommendation tasks. Furthermore, our analysis reveals that recommendation outcomes are **highly sensitive to text input**, highlighting the potential of prompt engineering in optimizing industrial-scale recommender systems.

1. Introduction

Recommender systems have become an integral part of people’s daily lives, revolutionizing the way users interact with content and services. To meet the diverse needs of users, recommender systems are evolving to become more sophisticated and multifaceted, leading to the emergence of a wide variety of recommendation tasks, each tailored to different aspects of user interaction and preference. For instance, due to the distribution shift of user behaviors in different contexts, such as in different apps or on different interaction interfaces within the same app, **multi-scenario tasks** are introduced for internal relationship modeling (Ma et al., 2018a). To precisely capture the user intention, **multi-objective tasks**, such as click prediction, purchase prediction, and like prediction, are formulated (Zheng & Wang, 2022). To prevent the phenomenon of information cocoons and offer novel candidates to users, specific tasks, such

as **serendipity recommendation** (Vanchinathan et al., 2014; Kotkov et al., 2023) and **long-tail item recommendation** (Liu & Zheng, 2020) are established. Additionally, there are occasions when it is necessary to model changes in user behavior over time, such as during seasonal changes, festival celebrations, or shopping events.

In traditional recommender systems, for each of the above tasks, a large amount of training data is collected, and specialized models are then designed, trained, evaluated, and deployed separately. This approach has served well to make progress on narrow tasks, but when the tasks change, it requires collecting new data and training a new model carefully, which is **time-consuming, lacks scalability, and sometimes faces challenges** due to insufficient task-specific data.

Multi-task learning (Caruana, 1997) is a promising framework for improving the versatility of recommender systems. However, learning multiple tasks simultaneously often leads to performance degradation compared to learning tasks individually, a phenomenon known as *negative transfer* (Zhang et al., 2022). In practice, it is necessary to design models according to the data size of each task. Tasks with less data should share more parameters to prevent insufficient training, while tasks with more data should have more independent parameters to avoid conflicts between tasks. Some studies have also proposed adaptive parameter sharing mechanisms (Ma et al., 2018a; Tang et al., 2020). However, the aforementioned methods rely heavily on human expertise, and as the number of tasks continues to increase, model design becomes increasingly challenging.

The previous challenges in multi-task learning arise because task conditioning has traditionally been implemented at the architectural level, requiring the model architecture of each task to be specifically adapted to its corresponding data. However, LLMs offer a new approach in which tasks can be defined through different prompts (Radford et al., 2018; 2019; Brown et al., 2020), rather than through specialized architectural modifications. In this paradigm, the inputs and outputs for different recommendation tasks follow a unified format, with the model architecture and parameters fully shared. Moreover, previous works (Kaplan et al., 2020; Hernandez et al., 2021) have shown that the issue of negative transfer can be efficiently mitigated by scaling up both the model and the data.

*Equal contribution ¹Taobao & Tmall Group of Alibaba, China. Correspondence to: Bo Zheng <bozheng@alibaba-inc.com>.

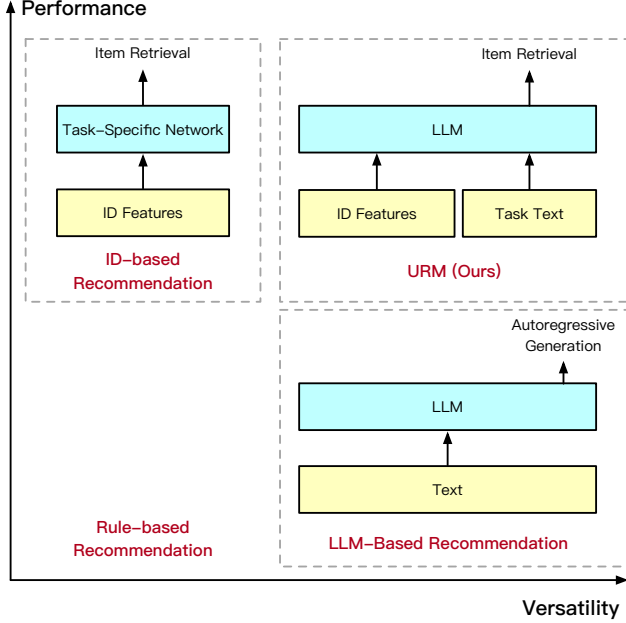


Figure 1. Comparison between different paradigms. Traditional ID-based recommendation models demonstrate high performance but are data-inefficient and labor-intensive when scaling to more tasks. Using LLMs directly for recommendation provides great task scalability, but the performance is often unsatisfactory and the efficiency is concerning. Our approach URM offers both high recommendation performance and task versatility.

Despite the excellent versatility of LLMs, a key challenge to use them in recommendation is how to improve performance and efficiency while retaining versatility. Recent methods try converting user behaviors to text (Geng et al., 2022b; Dai et al., 2023) or text embeddings (Chen et al., 2024; Jia et al., 2024), which, however, lack discriminability and may lead to poor performance in real systems. Furthermore, the token-by-token generation manner of LLMs involves multiple-time feedforward computing, which is intolerable in terms of latency and computational efficiency.

In this paper, we propose **Universal Recommendation Model (URM)**, which formulates multiple recommendation tasks that were previously studied in isolation into a unified input-output format using task prompts. By leveraging LLMs, URM enhances the versatility of recommendation models, enabling zero-shot task transfer and prompt tuning in industrial recommender systems.

To improve the effectiveness of LLM-based recommendation, we explore a new multimodal fusion module for item representation to leverage both the strong discriminability of ID embeddings and the rich semantics of text and image embeddings, which greatly improves the performance. Meanwhile, we design a sequence-in-set-out generation method, which allows the LLMs to generate a high-quality recommendation set in a single forward pass, enhancing the practicality of URM in industrial recommender systems.

We conduct comprehensive experiments across public and industrial-scale datasets. Both quantitative and qualitative experimental results demonstrate the excellent performance and versatility of our proposed URM.

2. Related Work

Multi-Task Learning. To effectively boost information sharing and alleviate task conflict, significant efforts have been invested in multi-task architecture designs. Existing techniques can be categorized into different parameter sharing methods (Misra et al., 2016; Kokkinos, 2017; Liu et al., 2019; Maninis et al., 2019; Heuer et al., 2021) and optimization strategies (Kendall et al., 2018; Chen et al., 2018; Yu et al., 2020). Multi-task learning has also been widely applied to recommender systems and achieved substantial improvement (Ma et al., 2018a;b; Tang et al., 2020). However, in practice, it is difficult to ensure that the performance of each task improves after multi-task learning, especially when the number of tasks continues to increase (Zamir et al., 2018; Jiang et al., 2023). Thus, it’s often necessary to carefully design the model structure based on the data proportions and the task relationships. This has also led to the isolated states among different recommendation problems, such as multi-scenario modeling (Jiang et al., 2022), multi-objective modeling (Ma et al., 2018b; Zheng & Wang, 2022), long-tail recommendation (Liu & Zheng, 2020), etc. In addition, search tasks can be viewed as a specialized type of recommendation with explicit query constraints. Due to significant distribution discrepancies and limited model capacity, traditional recommendation models cannot easily simultaneously handle scenarios both including and excluding explicit inputs (Liu et al., 2024b). In this paper, we propose to treat all the above problems as multi-task learning problems and address them simultaneously by LLMs, in an end-to-end and fully parameter-sharing manner.

LLMs for Recommendation. Large Language Models (LLMs) have demonstrated significant capabilities in natural language processing (Radford et al., 2018; 2019; Brown et al., 2020; Achiam et al., 2023), encouraging researchers to explore their application in recommender systems. Recent approaches treat recommendation tasks as natural language tasks, generating recommendations directly through prompting and in-context learning (Geng et al., 2022a; Dai et al., 2023; Liu et al., 2023; Zhang et al., 2023a; Kong et al., 2024). However, in real systems, users typically have hundreds or even millions of behaviors, leading to at least tens of thousands of text tokens, which increases inference costs and decreases LLM performance. Thus some methods introduce a hierarchical structure that encodes each item’s text or image information into item representations and feeds them into LLMs to generate a high-level user representation (Chen et al., 2024; Jia et al., 2024; Ye et al., 2024). Yet these representations still suffer from poor discriminability and

lead to low performance in industrial applications. Another approach employs traditional ID embeddings to represent items (Zhang et al., 2023b; Li et al., 2023; Liao et al., 2023), yet LLMs often struggle to interpret the intrinsic meaning of these embeddings. This limitation hinders the ability of LLMs to effectively adapt recommendations based on input prompts, thus degenerating their versatility. In this paper, we integrate the ID embeddings and text embedding within a single LLM to balance performance and versatility.

3. Approach

In URM, we propose to formulate recommendation tasks through LLM prompt templates to handle the multi-task learning problem. In this section, we first demonstrate the inputs and outputs of the LLM used in URM. Then, we illustrate the overall model architecture of URM, including the proposed multimodal fusion module and the sequence-in-set-out candidate generation pattern. Subsequently, we provide a detailed introduction on how to train URM.

3.1. Data Construction

Following the multi-task learning approach in the field of NLP (Radford et al., 2019), we utilize language to define recommendation tasks and represent inputs as sequences of symbols. Based on the characteristics of different tasks, we design multiple templates and then convert the industrial-scale user behavior data into sequence forms. Table 1 gives some examples of the prompt templates.

Input Format. To improve efficiency when handling long

Multi-scenario Recommendation: The items the user has recently clicked on are as follows: {USER BEHAVIOR SEQUENCE}. In scenario {SCENE}, please recommend items.

Multi-objective Recommendation: The items the user has recently clicked on are as follows: {USER BEHAVIOR SEQUENCE}. Please find items that the user will {ACTION}.

Long-tail Item Recommendation: The items the user has recently clicked on are as follows: {USER BEHAVIOR SEQUENCE}. Please recommend long-tail items.

Serendipity Recommendation: The items the user has recently clicked on are as follows: {USER BEHAVIOR SEQUENCE}. Please recommend some new item categories.

Long-term Recommendation: The items the user has recently clicked on are as follows: {USER BEHAVIOR SEQUENCE}. Please find items that match the user’s long-term interests.

Search Problem: The items the user has recently clicked on are as follows: {USER BEHAVIOR SEQUENCE}. Please recommend items that match {QUERY}.

Table 1. Examples of recommendation prompt templates. The different parts of each task are highlighted in gray. The computation of the shared parts can be accelerated using key-value caching, thus improving inference efficiency (Zheng et al., 2024).

user behavior sequences and maintain the discriminability of items, URM treats item as a kind of special token. The typical inputs are sentences composed of common text tokens and item IDs (such as [7502]) as follows:

Inputs: The items the user has recently clicked on are as follows: [7502][8308][8274][8380]. Please recommend items that match *Clothes*.

Output Format. Considering inference efficiency, URM is designed to generate items directly. Additionally, optimizing URM with target text is recommended to align with the semantic space and incorporate external textual knowledge. Typical targets are as follows:

Target Text: Swimwear & Beachwear for the Summer; Casual Dresses for Every Occasion.
Target Items: [3632][1334]

3.2. Model Architecture

URM is based on pre-trained LLMs due to their general capabilities across various tasks (Achiam et al., 2023). In Section 3.1, recommendation tasks have been converted into sequences consisting of text tokens and item IDs. Text tokens can be mapped to token embeddings by the vocabulary embedding table from the LLM. The number of item IDs in the industry can reach the billion range, thus we introduce a distributed item embedding module to convert each item ID into a unique item embedding. As shown in Figure 2, the embedding at each position in the sequence is then the sum of the position embedding and either the token embedding or the item embedding. The sequence’s embeddings are then processed through the LLM backbone, resulting in several hidden features that are used to compute the final output. To preserve the pre-training knowledge in the LLM, we retain the multi-layer transformer structure and only modify the structure of its input and output layers. We will elaborate on these aspects separately next.

Multimodal Fusion Module. There are currently two main methods to obtain item embeddings. The most common practice in the industry is to use real user behavior as a supervisory signal to learn a unique ID embedding for each item. However, the significant discrepancy between the embedding space of ID and that of text makes it challenging for LLMs to align the ID embeddings with their true meanings. As a result, LLMs reduce to functioning as similarity measures, losing versatility across different tasks. Another approach is to input the title (Jia et al., 2024), image (Ye et al., 2024), and other information into LLMs or content encoders to obtain the embedding for each item. Understanding the meaning of each item through text embedding is straightforward for LLMs. However, such item embeddings have poor discriminability, resulting in extremely bad

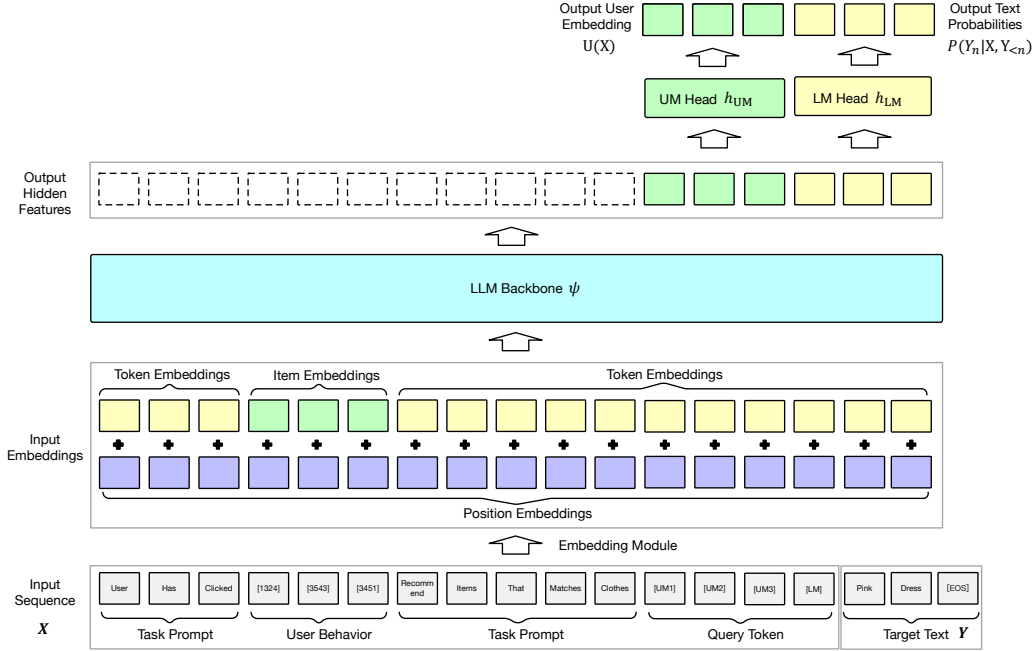


Figure 2. URM architecture. The input sequence consists of item IDs from user behavior, text tokens from task prompt, and several fixed query tokens, such as [UM] and [LM]. Item IDs are mapped to item embeddings by a distributed item embedding module and other tokens are mapped to token embeddings. The item embeddings or token embeddings are summed up with position embeddings, and fed into the LLM backbone. The outputs corresponding to the [UM] tokens are then mapped to the user representation space via the UM Head h_{UM} . The outputs corresponding to the [LM] token and its following tokens are mapped to the text space via the LM Head h_{LM} .

performance in practical industrial recommender systems.

To address the limitations of using ID or text embeddings alone, we design a multimodal fusion module to combine different types of item embeddings. As shown in Figure 3, the text and ID embeddings are all first transformed to the same dimensionality using an MLP layer. These embeddings are then combined by addition, followed by normalization using RMSNorm (Zhang & Sennrich, 2019), and processed through another MLP layer to produce the final multimodal item embedding. After supervised fine-tuning

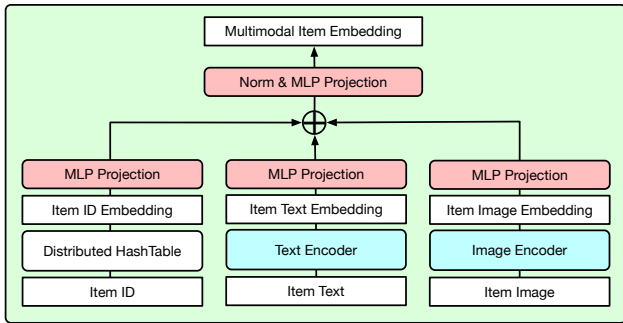


Figure 3. Multimodal Fusion Module. The parameters of the Text Encoder and the Image Encoder are frozen and linear projection layers are introduced to transform feature dimensions. Distributed HashTable is introduced for item IDs, which allows for industrial-scale IDs in the billion range. In practice, the item text embeddings and the corresponding image embeddings are precomputed and stored in a separate Distributed HashTable.

(SFT), the multimodal item fusion embeddings can effectively align with the textual semantic space while preserving each item’s specific information, striking a good balance between the generalization of text embeddings and the discriminability of ID embeddings. Note that this fusion approach is not limited to text embeddings. Semantic embeddings from other modalities, such as image embeddings (Sheng et al., 2024), can also be incorporated.

Sequence-In-Set-Out. Although the autoregressive generation manner (Radford et al., 2018; Geng et al., 2022a; Dai et al., 2023) is more versatile, it cannot ensure that the generated recommendation results are within a continuously updating candidate set. Furthermore, autoregressive generation of the sequence involves multiple forward inference costs of LLM, which is intolerable in terms of both latency and computational efficiency.

Therefore, we adopt special query tokens to generate target text and target item sets simultaneously. In our practice, sharing the *last_hidden_state* leads to poor results due to the conflicts between text and user’s representation spaces. Thus, we add special tokens [UM] and [LM] to the end of the input to indicate whether to output user representation or start generating text at a certain position, respectively. As for the output, we introduce two linear projections on the top layer features of the LLM, as shown in Figure 2. One head is the user modeling head h_{UM} to transform the LLM’s

hidden features into the user embeddings U , which is then used to retrieve relevant item sets during inference. The other one is the original language modeling head h_{LM} in the LLM, allowing URM to use text data for training.

Inputs: The items the user has recently clicked on are as follows: [7502][8308][8274][8380]. Please recommend items that match *Clothes*. [UM][LM]

By calculating the similarity between the generated user embeddings U and the multimodal item embeddings E , we can obtain the scores between the user and each item. The final objective during inference is to generate the top k item set from a large-scale corpus $\mathcal{C}$ with the largest scores,

$$\arg \text{Topk}_{Z \in \mathcal{C}} \mathcal{D}(U, E_Z), \quad (1)$$

where $\mathcal{D}$ is a similarity measure. Instead of measuring similarity with a single high-dimensional user embedding, we find that it is more effective to use multiple low-dimensional user embeddings, which is similar to the multi-head attention (Vaswani et al., 2017) architecture. Specifically, by increasing the number of [UM] tokens and reducing the output dimension of the UM head, URM can express different aspects of the target item set without increasing the total output dimensions. Without loss of generality, we take the inner product as the similarity measure for example. Denote the output user embeddings from LLM as $\mathbf{U} = (U_1, \dots, U_H)$, the multimodal embedding for item Z as E_Z , then the score between user and item Z is $\mathcal{D}(\mathbf{U}, E_Z) = \max(U_1 E_Z, \dots, U_H E_Z)$. In contrast to the sequential relationship among the auto-regressive generated tokens, the multiple [UM] tokens are fixed in the input sequence, thus a single LLM forward pass is sufficient to obtain the output for all [UM] tokens.

In industry, the size of the candidate set $\mathcal{C}$ can reach tens of millions or even hundreds of millions. Calculating the inner product for all candidates remains prohibitively expensive. Therefore, we construct an HNSW index (Malkov & Yashunin, 2016; Chen et al., 2022) based on the target multimodal embeddings and use hierarchical retrieval to output the final top k results. Our final inference cost for a single user primarily consists of once LLM feedforward inference plus the following $H \cdot \log(|\mathcal{C}|)$ times of inner product calculation in retrieval.

3.3. Training

Denote the input sequence as $\mathbf{X} = (X_1, \dots, X_M)$, the target text as $\mathbf{Y} = (Y_1, \dots, Y_N)$, and the target items as $\mathbf{Z} = (Z_1, \dots, Z_L)$. The text generation tasks can be optimized by the negative log-likelihood of the target text sequence:

$$\mathcal{L}_{\text{LM}}(\mathbf{X}, \mathbf{Y}) = - \sum_{n=1}^N \log P(Y_n | \mathbf{X}, \mathbf{Y}_{<n}), \quad (2)$$

where $P = \text{softmax}(h_{\text{LM}}(\psi(\cdot)))$ is the probabilities output by transformer ψ and the LM head h_{LM} . The item recommendation tasks are optimized by Noise Contrastive Estimation (NCE) Loss (Gutmann & Hyvärinen, 2010):

$$\mathcal{L}_{\text{UM}}(\mathbf{X}, \mathbf{Z}) = \sum_{l=1}^L - \log \frac{\exp(\mathcal{D}(\mathbf{U}(\mathbf{X}), E_{Z_l}))}{\sum_{Z \in \{Z_l\} \cup \mathcal{N}} \exp(\mathcal{D}(\mathbf{U}(\mathbf{X}), E_Z))} \quad (3)$$

where $\mathbf{U} = h_{\text{UM}}(\psi(\cdot))$ is the output by the transformer ψ and user modeling head h_{UM} . In each batch, negative examples $\mathcal{N}$ are sampled from the item candidates based on their occurrence frequency. The final training objective is

$$\min \mathcal{L}(\mathbf{X}, \mathbf{Y}, \mathbf{Z}) = \mathcal{L}_{\text{LM}}(\mathbf{X}, \mathbf{Y}) + \eta \mathcal{L}_{\text{UM}}(\mathbf{X}, \mathbf{Z}), \quad (4)$$

where η is the trade-off hyper-parameter. Considering the fact that URM has made significant modifications to both the input and output layers of the LLM, we adopt the full parameter SFT, with only the original embeddings of items, i.e., item ID embeddings, item text embeddings and item image embeddings being frozen.

4. Experiments

4.1. Public Dataset Experiments

Dataset. We first evaluate the performance of URM on Sports & Outdoors, Beauty and Toys & Games from Amazon Reviews (McAuley et al., 2015). We follow the preprocessing methods used in recent works (Geng et al., 2022a; Li et al., 2023) to construct the training and test datasets.

Models. For baseline, we use both traditional ID-based and LLM-based methods: (1) HGN (Ma et al., 2019) leverages a hierarchical structure to model user-item interactions. (2) GRU4Rec (Balázs Hidasi & Tikk, 2016) employs Gated Recurrent Units (GRUs) to model sequential user behavior. (3) Caser (Tang & Wang, 2018) leverages convolutional neural networks (CNNs) to capture sequential patterns in user behavior. (4) BERT4Rec (Fei Sun & Jiang, 2019) adapts the BERT architecture for sequential recommendation. (5) FDSA (Zhang et al., 2019) applies a self-attention module to model the relationships between features. (6) SASRec (Kang & McAuley, 2018) employs self-attention mechanisms to model user behavior sequences. (7) S3-Rec (Zhou et al., 2020) enhances sequential recommendation by incorporating self-supervised learning. (8) E4SRec (Li et al., 2023) integrates ID embedding within LLaMA2-13B (Touvron et al., 2023). (9) P5 (Geng et al., 2022b) employs the T5 (Sanh et al., 2022) model and takes recommendation tasks as purely natural language tasks.

For URM, we use Qwen-7B (Bai et al., 2023) as the LLM backbone. We use SASRec to generate 64-dimensional ID embeddings and use LLM to generate 4096-dimensional text embeddings (by feeding the item title into LLM, then

Table 2. Performance on Amazon dataset. RI: Relative Improvement.

Methods	Sports				Beauty				Toys			
	HR@5	NDCG@5	HR@10	NDCG@10	HR@5	NDCG@5	HR@10	NDCG@10	HR@5	NDCG@5	HR@10	NDCG@10
HGN	0.0189	0.0120	0.0313	0.0159	0.0325	0.0206	0.0512	0.0266	0.0321	0.0221	0.0497	0.0277
GRU4Rec	0.0129	0.0086	0.0204	0.0110	0.0164	0.0099	0.0283	0.0137	0.0097	0.0059	0.0176	0.0084
Caser	0.0116	0.0072	0.0194	0.0097	0.0205	0.0131	0.0347	0.0176	0.0166	0.0107	0.0270	0.0141
BERT4Rec	0.0115	0.0075	0.0191	0.0099	0.0203	0.0124	0.0347	0.0170	0.0116	0.0071	0.0203	0.0099
FDSA	0.0182	0.0122	0.0288	0.0156	0.0267	0.0163	0.0407	0.0208	0.0228	0.0140	0.0381	0.0189
SASRec	0.0233	0.0154	0.0350	0.0192	0.0387	0.0249	0.0605	0.0318	0.0445	0.0236	0.0698	0.0318
S3-Rec	0.0251	0.0161	0.0385	0.0204	0.0387	0.0244	0.0647	0.0327	0.0443	0.0294	0.0700	0.0376
E4SRec	0.0281	0.0196	0.0410	0.0237	0.0525	0.0360	0.0758	0.0435	0.0566	0.0405	0.0798	0.0479
P5	0.0387	0.0312	0.0460	0.0336	0.0508	0.0379	0.0664	0.0429	0.0648	0.0567	0.0709	0.0587
URM	0.0733	0.0488	0.1049	0.0590	0.0929	0.0671	0.1225	0.0766	0.0888	0.0619	0.1221	0.0726
RI	+89.4%	+56.4%	+128.0%	+75.6%	+77.0%	+77.0%	+61.6%	+76.1%	+37.0%	+9.2%	+53.0%	+23.7%

averaging and normalizing the last hidden features along the sequence dimension). The output user embedding dimension is set to 128 and the number of [UM] tokens is $H = 128$. The user behavior sequence length is 100 and trade-off hyper-parameter $\eta = 0.3$.

We evaluate the model performance using two widely adopted metrics, namely Hit Rate (HR@ K) and Normalized Discounted Cumulative Gain (NDCG@ K), computed at different ranking positions ($K = \{5, 10\}$) across the entire dataset. As presented in Table 2, URM outperforms the strongest baseline by an average of **74%** and **53%** in terms of HR@ K and nDCG@ K , respectively.

4.2. Industrial-scale Experiments

Dataset. Next, we verify the effectiveness of URM in an industrial-scale dataset, which is sampled from real traffic logs of the online system. The typical scenarios and objectives include Context-free Recommendation (CR), Recommendation for Scene A (RSA), Scene B (RSB), Scene C (RSC), Serendipity Recommendation (SR), Long-term Recommendation (LR), Long-tail Item Recommendation (LIR), Purchase Prediction (PP), and Search Problem (SP). This dataset contains hundreds of millions of samples and more than one billion distinct items in user behavior sequences. The candidate set contains tens of millions of items. A more detailed definition of the dataset is in Appendix A.1. We use samples from day 1 to day T for training and the samples of day $T + 1$ for testing. To improve training efficiency, we aggregate each user’s daily behaviors as the target set, which makes these recommendation tasks more challenging. All methods are compared in this setting to facilitate a more effective result.

Models. For baseline, we use the most commonly used and feasible methods in practice. (1) Two-tower Model, which uses two multi-layer MLPs to convert user-side and item-side features into embeddings, and use the inner product to obtain the final score. It’s the most widely used method in industry. (2) Transformer-based Model, which

uses a multi-layer transformer to convert user behavior sequence features into user embeddings (Zhai et al., 2024; Liu et al., 2024a). (3) Attention-DNN, which calculates cross-attention between the user behavior sequence and the target, and then uses multiple layers to calculate the score between the user and each item. It is typically used in the ranking stage (Zhou et al., 2018) and is a very strong baseline for candidate generation tasks (Zhu et al., 2019; Zhuo et al., 2020; Chen et al., 2022). For each method, we implement a single-task learning version (STL) with only task-specific data and a multi-task learning version (MTL) with data from all tasks. For the Attention-DNN model, which has the best performance among them, we further provide the Shared Bottom version (Ma et al., 2018b), the MMoE version (Ma et al., 2018a), and the PLE version (Tang et al., 2020). More details can be found in Appendix A.2.

We tuned the hyperparameters of each method including the embedding size and number of layers using the validation set. For URM, we only list the hyper-parameters that differ from those in Section 4.1. The ID Embeddings are generated by the Two-Tower Model optimized with only CR task. Considering that the dimension of LLM text embeddings is too high to store for a billion-scale item set, we use 128-dimensional semantic embeddings trained with CLIP contrastive learning (Radford et al., 2021) considering the image and text description of items as a substitute. The user behavior sequence length is 300 and the total length of text tokens, [UM] tokens, and item IDs is truncated to 1024.

We use recall to evaluate the effectiveness of our proposed method. Denote the output item set as $\mathcal{P}$ and the ground truth as $\mathcal{G}$, then recall R@K is

$$R@K = \frac{|\mathcal{P} \cap \mathcal{G}|}{|\mathcal{G}|}, \text{ where } |\mathcal{P}| = K \quad (5)$$

We show results in Table 3. Due to the complex task relationships, it is difficult for a single approach to perform well on all tasks, which leads to multi-task training being sometimes effective and sometimes causing negative transfer. In contrast, our proposed URM achieves the best performance

Table 3. Performance on the industrial dataset (metric: R@1000).

Model	Learning Method	CR	RSA	RSB	RSC	SR	LR	LIR	PP	SP	AVG
Two-tower Model	STL	0.129	0.271	0.166	0.129	0.069	0.066	0.117	0.146	0.355	0.161
	MTL	0.120	0.205	0.166	0.135	0.064	0.115	0.103	0.173	0.257	0.149
Transformer-based Model	STL	0.198	0.409	0.293	0.208	0.104	0.115	0.213	0.143	0.593	0.253
	MTL	0.192	0.390	0.319	0.221	0.076	0.218	0.207	0.401	0.744	0.308
Attention-DNN	STL	0.253	0.477	0.338	0.260	0.106	0.213	0.251	0.353	0.651	0.323
	MTL	0.238	0.456	0.375	0.277	0.062	0.336	0.265	0.478	0.671	0.351
	MTL-SharedBottom	0.243	0.442	0.376	0.270	0.072	0.337	0.224	0.505	0.745	0.357
	MTL-MMoE	0.233	0.439	0.375	0.257	0.070	0.325	0.218	0.491	0.736	0.349
	MTL-PLE	0.256	0.451	0.397	0.274	0.062	0.327	0.224	0.512	0.761	0.363
URM	MTL	0.263	0.530	0.439	0.362	0.093	0.285	0.240	0.581	0.835	0.403

in 6 out of 9 tasks and achieves a relative improvement of **11.0%** in average across all tasks. Besides, for the search problem, there is a specifically designed three-tower model, which adds an independent tower for queries and reaches 0.822 on SP R@1000. In contrast, our model achieves comparable results without any complex design. Experimental analysis of the inference efficiency and the results for other LLM backbones can be found in Appendix B.1 and B.2.

4.3. Ablation Study

The Effectiveness of Multi-Task Learning. Table 3 also gives the performance degradation of single-task learning compared to multi-task learning. Compared to other methods, URM performs better in multi-task settings, indicating that URM is less prone to negative transfer issues. Further, Figure 4 compares the performance of single-task and multi-task learning with different amounts of labeled data. The performance of URM on the single CR task can also continuously approach that of multi-task learning as the amount of task-specific data increases. However, URM with multi-task learning can converge faster to better performance using much less task-specific data, which has significant value for recommendation scenarios where a large amount of task-specific data is not available.

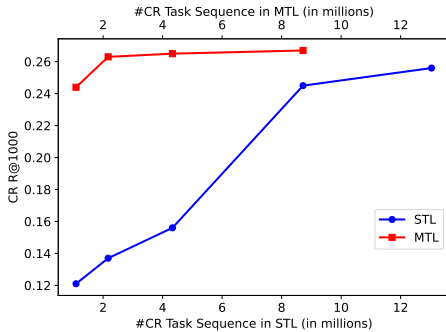


Figure 4. The effectiveness of multi-task learning.

The Effectiveness of Multimodal Item Representation. As shown in Table 4, when using fixed ID embedding only, the optimization potential of LLM is limited, resulting in

unremarkable performance on both the CR and SP tasks. On the other hand, when using only semantic embedding, it’s difficult to capture collaborative information, thus the model performs even worse. After using the multimodal fusion representation, LLM can balance semantic alignment and collaborative information, achieving improvements in both tasks. In Appendix B.5, we provide a visualization comparison of different item representations.

Table 4. The effect of item representation (metric: R@1000).

Methods	CR	SP
ID Embedding Only	0.170	0.663
Semantic Embedding Only	0.077	0.420
URM	0.263	0.835

The Effectiveness of Special Token. After removing the LM loss, as training progresses, the risk of an LLM forgetting the pre-trained parameters may increase. In Table 5, we can see that the item generation task is also negatively affected (URM w/o LM Loss). However, when the LM Loss is retained, if the [UM] token and [LM] token are not introduced for separate generations (URM w/o separated tokens in Table 5), the hidden features in the final layer may conflict with each other, leading to worse performance.

Table 5. The effectiveness of special token (metric: R@1000).

Methods	CR	SP
URM	0.263	0.835
URM w/o LM Loss	0.256	0.824
URM w/o Separated Tokens	0.072	0.528

We also verify the impact of the number of [UM] tokens. As shown in Figure 5, as the number of [UM] tokens increases, URM’s ability to represent the target item set becomes stronger, resulting in better performance.

4.4. Prompt Engineering

In this section, we explore the impact of different prompt templates during inference. Although we change the generation method to a sequence-in-set-out approach during inference for better performance and efficiency, we observe

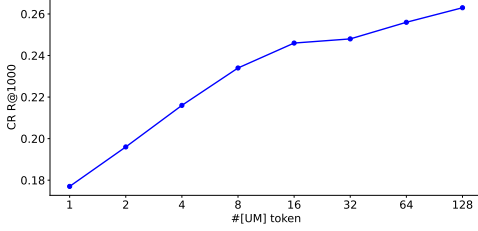


Figure 5. The impact of [UM] token number.

that LLM remains sensitive to the text input. This makes it possible to tune the distribution of the generated set through prompt engineering. Only two examples are provided here and more examples can be found in Appendix B.3.

Multi-Scene Recommendation. As shown in Table 6, with prompt templates for specific scenarios, URM aligns more closely with the data distribution and achieves higher performance, with a relative improvement (RI) of over 20%.

Table 6. The effect of prompts on multi-scenario recommendation.

Template	RSA R@1000	RSB R@1000	RSC R@1000
CR Template	0.440	0.335	0.278
RSA Template	0.530	0.409	0.240
RSB Template	0.522	0.439	0.257
RSC Template	0.444	0.327	0.362
RI	+20.5%	+31.0%	+30.2%

Serendipity Recommendation. This task requires finding items that users are interested in but the categories have not appeared in their behaviors. As illustrated in Table 7, after using the SR Template, the percent of the new category increased from 18.8% to 46.2%, leading to a relative increase of 82.3% in the SR R@1000.

Table 7. The effect of prompts on Serendipity Recommendation.

Template	CR R@1000	SR R@1000	Percent of New Category
CR Template	0.263	0.051	18.8%
SR Template	0.213	0.093	46.2%
RI	-	+82.3%	+145.7%

4.5. Zero-shot Task Transfer

In this section, we further explore the performance of URM when encountering new text and even new tasks.

New Query. We analyze the performance of URM on seen and unseen queries. Note that the distributions of these two types of queries are different and unseen queries often tend to be long-tail queries. To make a more reasonable comparison, we plot the model’s performance on seen and unseen queries as the query frequency increases. Figure 6 shows that URM demonstrates strong robustness to query input, indicating that URM can generalize well even after

SFT on a small dataset. Meanwhile, tasks that are closely related to user needs, can be expressed by modifying the query in the SP Template and can be directly transferred. Some specific cases are provided in the Appendix B.4.

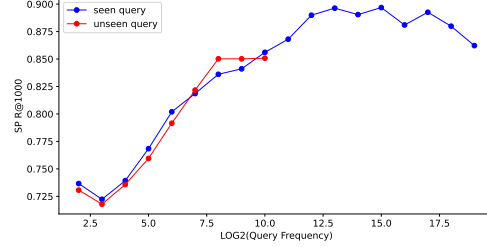


Figure 6. Comparison of performance on seen and unseen queries.

New Prompt Template. We find that the URM can transfer to tasks with hybrid prompt templates, such as long-tail item recommendations for search.

Inputs: The items the user has recently clicked on are as follows: {USER BEHAVIOR SEQUENCE}. Please recommend long-tail items that match {QUERY}.

As shown in Table 8, when the SP template and LIR template are combined, the set produced by the URM aligns better with the query, thereby improving search performance compared to using the LIR template. At the same time, the proportion of long-tail items in the output set also increases compared to that of the SP template. More cases can be found in Appendix B.4. Such task versatility is actually difficult for traditional ID-based recommendation models to achieve. To further improve the transferability of URM to any prompt template, it is necessary to construct a richer set of training prompt templates, thereby enabling URM to build a comprehensive mapping relationship between a given sequence and a target set.

Table 8. The result of hybridizing the SP and LIR templates.

Template	SP R@1000	Percent of Long-tail Items
SP Template	0.835	79.6%
LIR Template	0.630	81.6%
SP × LIR Template	0.836	82.4%

5. Conclusion

Our proposed URM effectively integrates multiple recommendation tasks into a unified framework, leveraging LLMs for enhanced versatility and performance across various datasets. Furthermore, the incorporation of multimodal fusion representation and sequence-in-set-out generation method significantly improves the practicality and efficacy of URM in industrial recommender systems.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

Acknowledgements

We sincerely appreciate Zhengxiong Zhou, Zhengyu Liu, Qingzhang Shi, Zexin Yan, Jianqi Xie, Zequn Wu, Jinjing Li, Qian Wang, Yunlong Xu for implementing the key components of the training and serving infrastructure.

References

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Bai, J., Bai, S., Chu, Y., Cui, Z., Dang, K., Deng, X., Fan, Y., Ge, W., Han, Y., Huang, F., Hui, B., Ji, L., Li, M., Lin, J., Lin, R., Liu, D., Liu, G., Lu, C., Lu, K., Ma, J., Men, R., Ren, X., Ren, X., Tan, C., Tan, S., Tu, J., Wang, P., Wang, S., Wang, W., Wu, S., Xu, B., Xu, J., Yang, A., Yang, H., Yang, J., Yang, S., Yao, Y., Yu, B., Yuan, H., Yuan, Z., Zhang, J., Zhang, X., Zhang, Y., Zhang, Z., Zhou, C., Zhou, J., Zhou, X., and Zhu, T. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.
- Balázs Hidasi, Alexandros Karatzoglou, L. B. and Tikk, D. Session-based recommendations with recurrent neural networks. In *ICLR*, 2016.
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. Language models are few-shot learners. In *NeurIPS*, 2020.
- Caruana, R. Multitask learning. *Machine Learning*, 28: 41–75, 1997.
- Chen, J., Chi, L., Peng, B., and Yuan, Z. Hllm: Enhancing sequential recommendations via hierarchical large language models for item and user modeling. *arXiv preprint arXiv:2409.12740*, 2024.
- Chen, R., Liu, B., Zhu, H., Wang, Y., Li, Q., Ma, B., Hua, Q., Jiang, J., Xu, Y., Deng, H., et al. Approximate nearest neighbor search under neural similarity metric for large-scale recommendation. In *CIKM*, 2022.
- Chen, Z., Badrinarayanan, V., Lee, C.-Y., and Rabinovich, A. Gradnorm: Gradient normalization for adaptive loss balancing in deep multitask networks. In *ICML*, 2018.
- Dai, S., Shao, N., Zhao, H., Yu, W., Si, Z., Xu, C., Sun, Z., Zhang, X., and Xu, J. Uncovering chatgpt’s capabilities in recommender systems. *Proceedings of the 17th ACM Conference on Recommender Systems*, 2023.
- Dao, T. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.
- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Fei Sun, Jun Liu, J. W. C. P. X. L. W. O. and Jiang, P. Bert4rec: Sequential recommendation with bidirectional encoder representations from transformer. In *CIKM*, 2019.
- Geng, S., Liu, S., Fu, Z., Ge, Y., and Zhang, Y. Recommendation as language processing (RLP): A unified pretrain, personalized prompt & predict paradigm (P5). In *RecSys*, 2022a.
- Geng, S., Liu, S., Fu, Z., Ge, Y., and Zhang, Y. Recommendation as language processing (rlp): A unified pretrain, personalized prompt & predict paradigm (p5). In *Proceedings of the Sixteenth ACM Conference on Recommender Systems*, 2022b.
- Gutmann, M. and Hyvärinen, A. Noise-contrastive estimation: A new estimation principle for unnormalized statistical models. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pp. 297–304. JMLR Workshop and Conference Proceedings, 2010.
- Hernandez, D., Kaplan, J., Henighan, T., and McCandlish, S. Scaling laws for transfer. *arXiv preprint arXiv:2102.01293*, 2021.
- Heuer, F., Mantowsky, S., Bukhari, S., and Schneider, G. Multitask-centernet (mcn): Efficient and diverse multitask learning using an anchor free approach. In *ICCV*, 2021.
- Jia, J., Wang, Y., Li, Y., Chen, H., Bai, X., Liu, Z., Liang, J., Chen, Q., Li, H., Jiang, P., and Gai, K. Knowledge adaptation from large language model to recommendation for practical industrial application. *ArXiv*, abs/2405.03988, 2024.
- Jiang, J., Chen, B., Pan, J., Wang, X., Dapeng, L., Jiang, J., and Long, M. Forkmerge: Mitigating negative transfer in auxiliary-task learning. In *NeurIPS*, 2023.

- Jiang, Y., Li, Q., Zhu, H., Yu, J., Li, J., Xu, Z., Dong, H., and Zheng, B. Adaptive domain interest network for multi-domain recommendation. *arXiv preprint arXiv:2206.09672*, 2022.
- Kang, W.-C. and McAuley, J. J. Self-attentive sequential recommendation. In *ICDM*, 2018.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- Kendall, A., Gal, Y., and Cipolla, R. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In *CVPR*, 2018.
- Kokkinos, I. Ubertnet: Training a ‘universal’ convolutional neural network for low-, mid-, and high-level vision using diverse datasets and limited memory. In *CVPR*, 2017.
- Kong, X., Wu, J., Zhang, A., Sheng, L., Lin, H., Wang, X., and He, X. Customizing language models with instance-wise lora for sequential recommendation. In *NeurIPS*, 2024.
- Kotkov, D., Medlar, A., and Glowacka, D. Rethinking serendipity in recommender systems. In *SIGIR*, 2023.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- Langley, P. Crafting papers on machine learning. In Langley, P. (ed.), *Proceedings of the 17th International Conference on Machine Learning (ICML 2000)*, pp. 1207–1216, Stanford, CA, 2000. Morgan Kaufmann.
- Li, X., Chen, C., Zhao, X., Zhang, Y., and Xing, C. E4srec: An elegant effective efficient extensible solution of large language models for sequential recommendation. *CoRR*, abs/2312.02443, 2023.
- Liao, J., Li, S., Yang, Z., Wu, J., Yuan, Y., and Wang, X. Llara: Aligning large language models with sequential recommenders. *CoRR*, abs/2312.02445, 2023.
- Liu, C., Cao, J., Huang, R., Zheng, K., Luo, Q., Gai, K., and Zhou, G. Kuaiformer: Transformer-based retrieval at kuaishou, 2024a. URL <https://arxiv.org/abs/2411.10057>.
- Liu, J., Liu, C., Lv, R., Zhou, K., and Zhang, Y. Is chatgpt a good recommender? A preliminary study. *CoRR*, abs/2304.10149, 2023.
- Liu, J., Chen, Q., Xu, J., Li, J., Li, B., and Xu, S. A unified search and recommendation framework based on multi-scenario learning for ranking in e-commerce. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 2890–2894, 2024b.
- Liu, S. and Zheng, Y. Long-tail session-based recommendation. In *Proceedings of the 14th ACM Conference on Recommender Systems*, 2020.
- Liu, S., Johns, E., and Davison, A. J. End-to-end multi-task learning with attention. In *CVPR*, 2019.
- Long, M., Cao, Y., Wang, J., and Jordan, M. I. Learning transferable features with deep adaptation networks. In *ICML*, 2015.
- Ma, C., Kang, P., and Liu, X. Hierarchical gating networks for sequential recommendation. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2019, Anchorage, AK, USA, August 4-8, 2019*, 2019.
- Ma, J., Zhao, Z., Yi, X., Chen, J., Hong, L., and Chi, E. H. Modeling task relationships in multi-task learning with multi-gate mixture-of-experts. In *SIGKDD*, 2018a.
- Ma, X., Zhao, L., Huang, G., Wang, Z., Hu, Z., Zhu, X., and Gai, K. Entire space multi-task model: An effective approach for estimating post-click conversion rate. In *SIGIR*, 2018b.
- Malkov, Y. and Yashunin, D. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2016.
- Maninis, K.-K., Radosavovic, I., and Kokkinos, I. Attentive single-tasking of multiple tasks. In *CVPR*, 2019.
- McAuley, J. J., Targett, C., Shi, Q., and van den Hengel, A. Image-based recommendations on styles and substitutes. In *SIGIR*, 2015.
- Misra, I., Shrivastava, A., Gupta, A., and Hebert, M. Cross-stitch networks for multi-task learning. In *CVPR*, 2016.
- Radford, A., Narasimhan, K., Salimans, T., and Sutskever, I. Improving language understanding by generative pre-training. *Technical report, OpenAI*, 2018.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I. Language models are unsupervised multitask learners. *Technical report, OpenAI*, 2019.
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., et al. Learning transferable visual models from natural language supervision. In *ICML*, 2021.

- Sanh, V., Webson, A., Raffel, C., Bach, S. H., Sutawika, L., Alyafeai, Z., Chaffin, A., Stiegler, A., Raja, A., Dey, M., Bari, M. S., Xu, C., Thakker, U., Sharma, S. S., Szczechla, E., Kim, T., Chhablani, G., Nayak, N. V., Datta, D., Chang, J., Jiang, M. T., Wang, H., Manica, M., Shen, S., Yong, Z. X., Pandey, H., Bawden, R., Wang, T., Neeraj, T., Rozen, J., Sharma, A., Santilli, A., Févry, T., Fries, J. A., Teehan, R., Scao, T. L., Biderman, S., Gao, L., Wolf, T., and Rush, A. M. Multitask prompted training enables zero-shot task generalization. In *ICLR*, 2022.
- Sheng, X.-R., Yang, F., Gong, L., Wang, B., Chan, Z., Zhang, Y., Cheng, Y., Zhu, Y.-N., Ge, T., Zhu, H., et al. Enhancing taobao display advertising with multimodal representations: Challenges, approaches and insights. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, pp. 4858–4865, 2024.
- Tang, H., Liu, J., Zhao, M., and Gong, X. Progressive layered extraction (ple): A novel multi-task learning (mtl) model for personalized recommendations. In *RecSys*, 2020.
- Tang, J. and Wang, K. Personalized top-n sequential recommendation via convolutional sequence embedding. In *WSDM*, 2018.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., and Lample, G. Llama: Open and efficient foundation language models. *CoRR*, abs/2302.13971, 2023.
- Vanchinathan, H. P., Nikolic, I., De Bona, F., and Krause, A. Explore-exploit in top-n recommender systems via gaussian processes. In *Proceedings of the 8th ACM Conference on Recommender Systems*, 2014.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. In *NeurIPS*, 2017.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022.
- Ye, Y., Zheng, Z., Shen, Y., Wang, T., Zhang, H., Zhu, P., Yu, R., Zhang, K., and Xiong, H. Harnessing multimodal large language models for multimodal sequential recommendation. *arXiv preprint arXiv:2408.09698*, 2024.
- Yu, T., Kumar, S., Gupta, A., Levine, S., Hausman, K., and Finn, C. Gradient surgery for multi-task learning. In *NeurIPS*, 2020.
- Zamir, A. R., Sax, A., Shen, W. B., Guibas, L. J., Malik, J., and Savarese, S. Taskonomy: Disentangling task transfer learning. In *CVPR*, 2018.
- Zhai, J., Liao, L., Liu, X., Wang, Y., Li, R., Cao, X., Gao, L., Gong, Z., Gu, F., He, J., Lu, Y., and Shi, Y. Actions speak louder than words: Trillion-parameter sequential transducers for generative recommendations. In *ICML*, 2024.
- Zhang, B. and Sennrich, R. Root mean square layer normalization. In *NeurIPS*, 2019.
- Zhang, J., Xie, R., Hou, Y., Zhao, W. X., Lin, L., and Wen, J. Recommendation as instruction following: A large language model empowered recommendation approach. *CoRR*, abs/2305.07001, 2023a.
- Zhang, T., Zhao, P., Liu, Y., Sheng, V. S., Xu, J., Wang, D., Liu, G., and Zhou, X. Feature-level deeper self-attention network for sequential recommendation. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI 2019, Macao, China, August 10-16, 2019*. ijcai.org, 2019.
- Zhang, W., Deng, L., Zhang, L., and Wu, D. A survey on negative transfer. *IEEE/CAA Journal of Automatica Sinica*, 2022.
- Zhang, Y., Feng, F., Zhang, J., Bao, K., Wang, Q., and He, X. Collm: Integrating collaborative embeddings into large language models for recommendation. *CoRR*, abs/2310.19488, 2023b.
- Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. Sglang: Efficient execution of structured language model programs, 2024. URL <https://arxiv.org/abs/2312.07104>.
- Zheng, Y. and Wang, D. X. A survey of recommender systems with multi-objective optimization. *Neurocomputing*, 2022.
- Zhou, G., Zhu, X., Song, C., Fan, Y., Zhu, H., Ma, X., Yan, Y., Jin, J., Li, H., and Gai, K. Deep interest network for click-through rate prediction. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 1059–1068, 2018.
- Zhou, K., Wang, H., Zhao, W. X., Zhu, Y., Wang, S., Zhang, F., Wang, Z., and Wen, J.-R. S3-rec: Self-supervised learning for sequential recommendation with mutual information maximization. In *Proceedings of the 29th ACM international conference on information & knowledge management*, pp. 1893–1902, 2020.

Zhu, H., Li, X., Zhang, P., Li, G., He, J., Li, H., and Gai, K. Learning tree-based deep model for recommender systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 1079–1088, 2018.

Zhu, H., Chang, D., Xu, Z., Zhang, P., Li, X., He, J., Li, H., Xu, J., and Gai, K. Joint optimization of tree-based index and deep model for recommender systems. *Advances in Neural Information Processing Systems*, 32, 2019.

Zhuo, J., Xu, Z., Dai, W., Zhu, H., Li, H., Xu, J., and Gai, K. Learning optimal tree models under beam search. In *International Conference on Machine Learning*, pp. 11650–11659. PMLR, 2020.

A. Implementation Details

A.1. Dataset Details

We construct datasets from typical scenarios and objectives, listed as follows:

- **Context-free Recommendation (CR)**: Recommend items to users without constraints. Positive samples are click behaviors across all scenarios or contexts.
- **Recommendation for Scene A (RSA), Scene B (RSB), Scene C (RSC)**: Positive samples are the items exposed to users in 3 scenarios with distribution shift.
- **Serendipity Recommendation (SR)**: Recommend new items to users to mitigate the information cocoon effect. Positive samples are real click behaviors, whose categories have not appeared in user behaviors.
- **Long-term Recommendation (LR)**: Recommend items based on long-term behaviors of users. Positive samples are the users' real click behaviors, whose categories only appear in the long-term user behavior sequence.
- **Long-tail Item Recommendation (LIR)**: Recommend long-tail items to mitigate the bias of the recommender system. Positive samples are the items that users have clicked, and the popularity of which is below a certain threshold.
- **Purchase Prediction (PP)**: Positive samples are the items that the users have purchased.
- **Search Problem (SP)**: Given a query, recommend the most relevant items that the user is most likely to be interested in. Positive samples are the user's click behaviors under a certain query.

This dataset contains billions of training samples in total. For URM, it is quite difficult to consume a vast amount of recommendation data in a short time. Therefore, we sample 12.7 million training examples from the whole dataset. The amount of sampled training data for each task is shown in Table 9.

Table 9. Statistics of the sampled training dataset.

Task	#Behaviors
Context-free Recommendation (CR)	3,146,887
Recommendation for Scene A (RSA)	559,449
Recommendation for Scene B (RSB)	557,237
Recommendation for Scene C (RSC)	1,331,326
Serendipity in Recommendation (SR)	1,097,952
Long-term Recommendation (LR)	1,169,954
Long-tail Item Recommendation (LIR)	867,462
Purchase Prediction (PP)	103,205
Search Problem (SP)	3,909,578

A.2. Model Implementation Details

Two-tower Model. Both the user-side features and the item-side features are flattened to 2-dimensional, and then fed into 3-layer MLP networks with layers [256, 128, 64] to obtain the final 64-dimensional user embeddings and item embeddings. The user embeddings will not be normalized, while the item embeddings will be normalized.

Transformer-based Model. After adding the position embedding, the user behavior sequence embedding, task embedding, and query embedding are concatenated and fed into a 5-layer Transformer network, where each Transformer layer has 4 heads and the dimension of the FFN layer is 4 times larger than the input dimension. The embedding at the last position is taken as the user embedding. The item side follows the same as the two-tower model: features are flattened and then processed through a three-layer MLP network with dimensions [256, 128, 64] to obtain the final 64-dimensional embedding, which is then normalized.

Attention-DNN. The user side includes multiple aggregated features and 3 sequence features of different lengths. The sequence features are used to compute cross attention with the item features (Zhou et al., 2018). The item side also includes multiple features. After flattening all features, they are collectively fed into a fully connected network with layers [256, 128, 64, 1] to obtain the final score. Due to the increase in computation cost for each item, we build HNSW index (Malkov & Yashunin, 2016) on the item embeddings for faster inference following (Zhu et al., 2018; Chen et al., 2022).

Attention-DNN + Shared Bottom. Retain the attention module and all features in Attention-DNN, and use different MLPs for modeling each task separately.

Attention-DNN + MMoE. Retain the attention module and all features in Attention-DNN, and introduce 4 MLPs to generate the middle representations. The task embedding is fed into the gate network to obtain combination weights, which are then used to adaptively combine the outputs from the MLP.

Attention-DNN + PLE. Retain the attention module and all features in Attention-DNN and adopt 4 MLPs for shared module and task-specific module separately to generate the middle representations with the dim of 64. These middle representations of the 4 shared modules and each task-specific module are then merged using a gate network and fed into each task’s 2-layered MLPs to obtain the final score.

URM. As illustrated in Figure 3, URM first employs 2 linear layers to project item semantic embeddings and item ID embeddings into 128-dimensional embeddings. These embeddings are then summed and passed through an RMSNorm layer, a linear layer, and a normalization layer to obtain the final 128-dimensional multimodal item embeddings. An additional linear layer will map the multimodal item embeddings into a 4096-dimensional tensor. Simultaneously, the prompt text is also transformed into this dimension by the token embedding table. As depicted in Figure 2, once merged with positional embeddings, the input embeddings are fed into the LLM backbone. The hidden state from the final transformer layer will subsequently pass through an MLP layer and yield the final, non-normalized user embedding of size $H \times 128$.

A.3. Training Details

For URM, we use AdamW with an initial learning rate of $2e^{-5}$, weight decay of 0.05, batch size of 4096, and cosine learning rate decay. Each batch consists of 10,000 negative examples, which are sampled from the item candidates according to the $3/4$ power of their occurrence frequency. The sampled training dataset is trained for a single epoch. To preserve the pre-trained knowledge in LLM, we follow transfer learning (Long et al., 2015) and set the learning rate of the pre-trained layers to be $1/10$ of the learning rate of the other layers. Using Qwen-7B as the LLM backbone on 64 NVIDIA H20 GPUs, it takes 33 hours to process 12.7 million training examples.

B. More Experimental Results

B.1. Analysis of Inference Efficiency

In the inference stage, our model architecture differs from conventional LLMs in two aspects: (1) the multimodal item embedding module and (2) the item retrieval module. Both modules are well-established within ID-based recommendation models and leverage mature, high-performance technologies. Specifically, for the multimodal item embedding module, we utilize distributed hashtable lookup technology to reduce the embedding retrieval latency to within a few milliseconds. For the item retrieval module, we employ HNSW retrieval, enabling efficient searches within a ten-million-item database in under 10 milliseconds. Consequently, the negative impact of these two additional modules on performance is negligible, making our model nearly equivalent to an LLM inference with an output token length of 1. This enables seamless utilization of existing inference frameworks (e.g., vLLM (Kwon et al., 2023)) and acceleration techniques (including batching, key-value caching, FlashAttention (Dao et al., 2022; Dao, 2024), etc.).

We evaluated inference speed on NVIDIA H20 GPUs using Qwen-7B as the base model. With batch size=4 and input token length about 1024, our system achieves batch latency of 220ms, single-GPU QPS of 20, and daily throughput of 1.7 million requests per GPU. In contrast, traditional LLM-based recommendation methods, which require auto-regressively generating multiple tokens, result in online latencies on the order of seconds and QPS less than 1, significantly underperforming compared to our approach.

B.2. Experiments on More LLM Backbones

We also conduct experiments on Qwen-1.8B. As shown in Table 10, Qwen-1.8B performs worse than Qwen-7B on most tasks, which indicates that larger-scale parameters also lead to better generalizability in recommendation tasks.

Table 10. Comparison between Qwen-7B and 1.8B (metric: R@1000).

Methods	CR	RSA	RSB	RSC	SR	LR	LIR	PP	SP	AVG
Qwen-7B	0.263	0.530	0.439	0.362	0.093	0.285	0.240	0.581	0.835	0.403
Qwen-1.8B	0.255	0.529	0.434	0.351	0.081	0.243	0.230	0.572	0.835	0.392

B.3. More Experiments on Prompt Engineering

Long-tail Item Recommendation. As shown in Table 11, after using the LIR Template, the proportion of long-tail items in the recommendation set increases by 49.5% relatively.

Table 11. Effect of prompts on Long-tail Item Recommendation.

Template	CR R@1000	LIR R@1000	Percent of Long-tail Items
CR Template	0.263	0.240	54.6%
LIR Template	0.202	0.240	81.6%
RI	-	+0%	+49.5%

Long-term Recommendation. As shown in Table 12, employing the LR Template results in a relative increase of 96.6% in the proportion of items within the recommendation set that align with the user’s long-term interests and a relative increase of 36.4% in LR R@1000.

Table 12. Effect of prompts on Long-term Recommendation.

Template	CR R@1000	LR R@1000	Percent of Long- term Interest
CR Template	0.263	0.209	32.6%
LR Template	0.149	0.285	64.1%
RI	-	+36.4%	+96.6%

Diversity of Recommendation Result. In some of the training data, we inject the number of categories in the target set into the prompt as shown below.

Inputs: The items the user has recently clicked on are as follows: {USER BEHAVIOR SEQUENCE}. Please recommend the top $\{K\}$ categories that users are most likely to be interested in. [UM][LM]

Then, during the test stage, we observe that by adjusting the size of K in the prompt, we could modify the category diversity in the recommendation set as shown in Table 13.

Table 13. Effect of prompts on recommendation diversity.

K	Category Recall@1000	#Category
4	0.767	74.7
8	0.772	80.8
16	0.775	94.1
32	0.777	104.2
64	0.778	113.7
128	0.780	117.1
RI	+1.7%	+56.8%

B.4. More Experiments on Zero-shot Task Transfer

New Prompt: Serendipity & Purchase Prediction. As shown in Table 14, when the SR template and PP template are combined, the set produced by the URM aligns better with the purchase objective, thereby improving PP R@1000 compared

to using the SR template. At the same time, the percentage of new categories in the output set also increases compared to that of the PP template. This makes the recommended set close to the distribution of items that the user has not clicked on before but is highly likely to purchase next.

Table 14. The result of hybridizing the SR and PP templates.

Metric	PP R@1000	Percent of New Category
PP Template	0.581	25.6%
SR Template	0.411	46.2%
PP $\times$ SR Template	0.510	45.1%

New Prompt: Long-tail Item Recommendation for Scene A. As shown in Table 15, when the RSA template and LIR template are combined, the output set is closer to the distribution of scenario A, thereby improving RSA R@1000 compared to using the LIR template. Additionally, there is an increase in the proportion of long-tail items within the output set compared to the RSA template alone. Consequently, the recommended set becomes more representative of the long-tail items that are most likely to occur in scenario A.

Table 15. The result of hybridizing the RSA and LIR templates.

Metric	RSA R@1000	Percent of Long-tail Items
RSA Template	0.530	72.6%
LIR Template	0.410	81.6%
RSA $\times$ LIR Template	0.483	81.5%

New Prompt. Further, we validate the zero-shot task transfer performance from the CR task to the SP task. As shown in Figure 7, as the number of training samples increases, URM trained only on the CR task can still learn the mapping between the query and target sets and reach 0.698 on SP R@1000! The reason is that our training loss includes the language modeling loss, which allows the URM to align item embeddings with the semantic space, even when no query-related tasks exist in the training samples. Of course, the performance of zero-shot task transfer still has a significant gap with supervised training on the corresponding task (MTL vs STL on CR). However, this fully demonstrates that URM inherits the powerful task transfer capabilities of LLM and also underscores the importance of retaining LM loss.

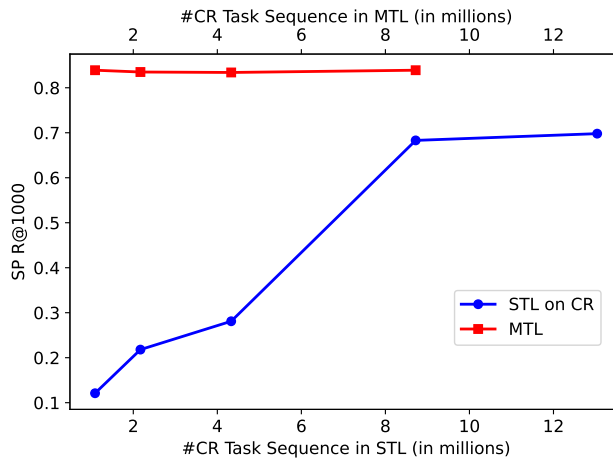


Figure 7. Zero-shot task transfer performance from the CR task to the SP task.

New Context. By modifying the query in the SP prompt to a specific context, we can model the changes in user behavior over periods, such as during seasonal changes, festival celebrations, or shopping events. Figure 8 gives two examples. The

fundamental reason why URM can achieve this is that the training data for the SP (search problem) has established a broad mapping relationship between user behaviors, text constraints, and the target item set. This has two potential advantages.

1. By changing the query to a certain context, we can inject external world knowledge into URM, allowing the recommendation results to get ready for potential upcoming events.
2. This also supports the combination of URM and Chain-of-Thought (CoT) technologies (Wei et al., 2022). Specifically, the LLM generates intermediate results in the form of text through reasoning, and then injects this text as context into URM, thereby producing the final set.

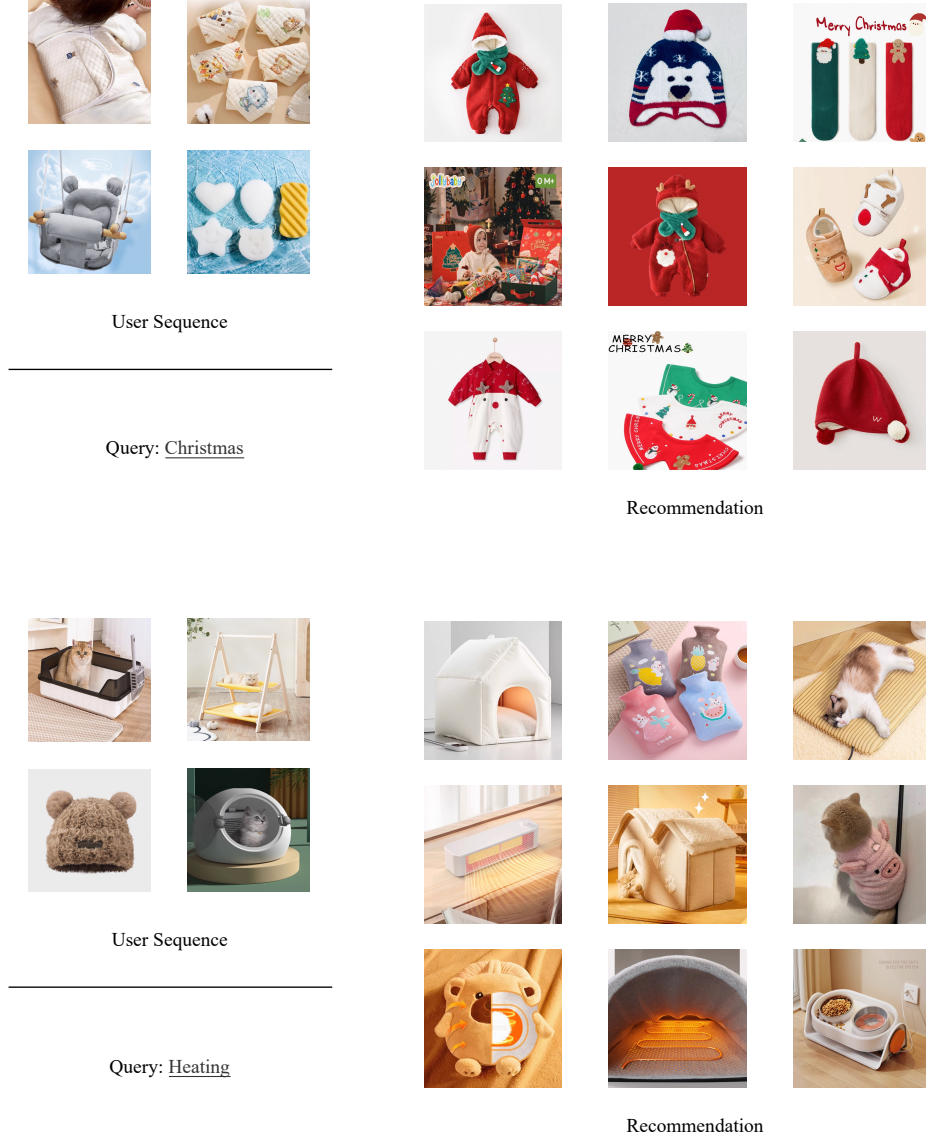


Figure 8. Zero task transfer to a new context. Christmas is a festive celebration, and heating is a common need in winter. When these two contexts are injected as queries into the prompt, the model can provide recommendations based on the user’s historical behavior and the given context simultaneously.

B.5. Visualization Comparison of Different Item Representation

To visualize different item representations, we use the nearest neighbor retrieval approach to find similar items of a given item under a specific representation. As shown in Figure 9,10, semantic embeddings mainly capture similarity relationships, whereas ID embeddings are more centered on co-occurrence relationships. The multimodal fusion representations fall somewhere in between these two.

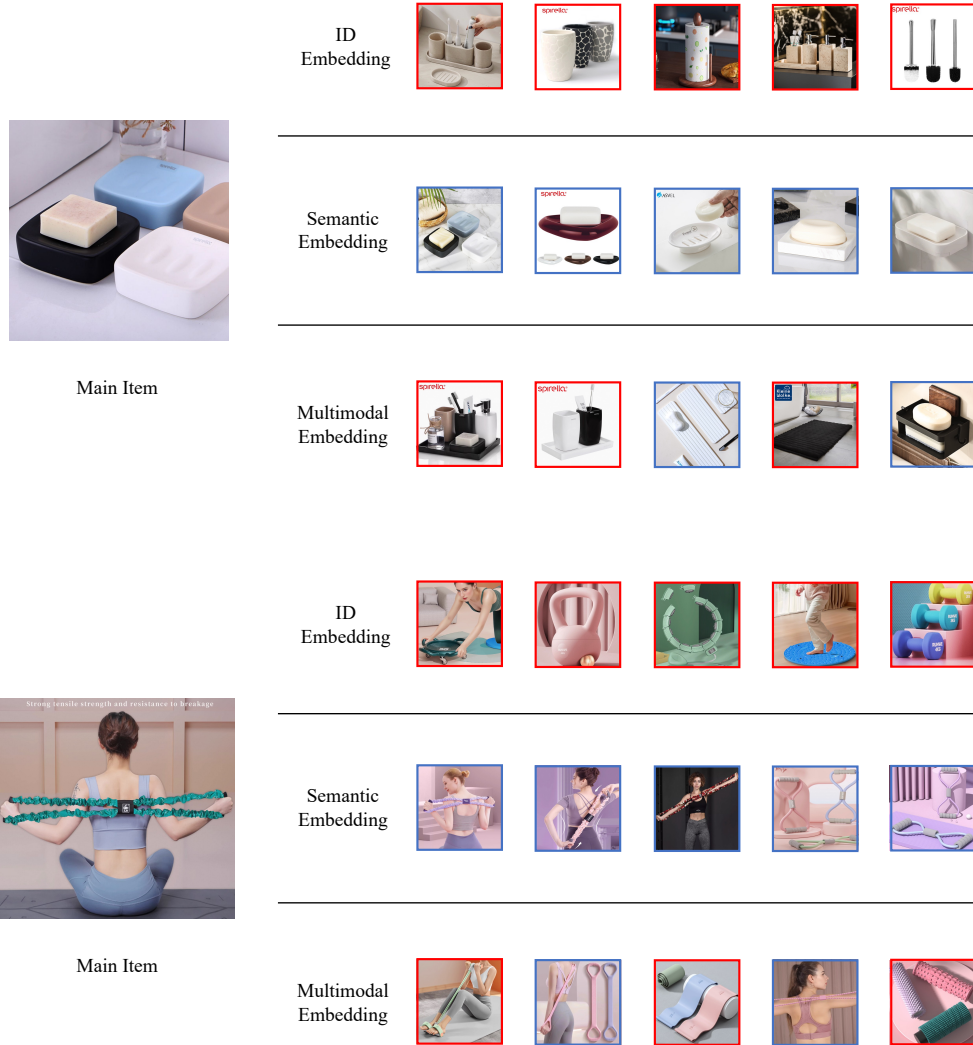


Figure 9. Visualization of different item representations

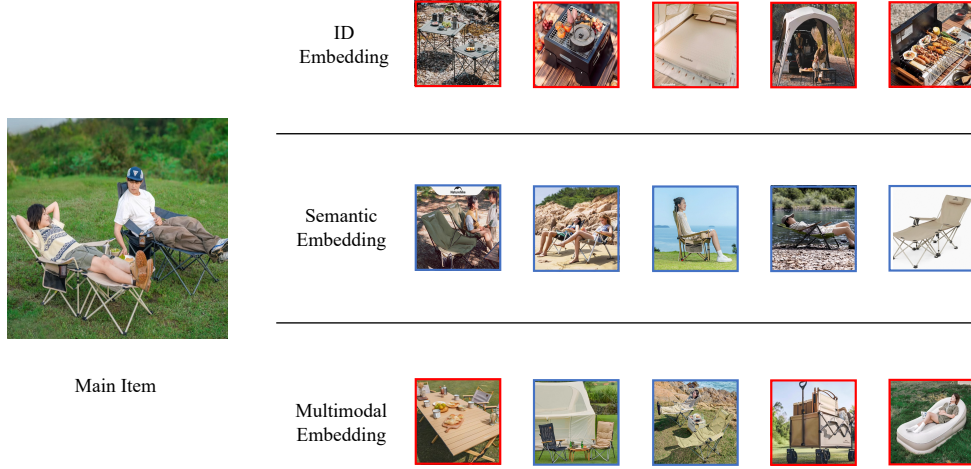


Figure 10. Visualization of different item representations

B.6. More Experiments on the Effect of [UM] Token

Visualization of User Embeddings from Different [UM] Tokens. To further validate the role of the [UM] token, we categorize the output 1000 items from URM into the user embeddings that have the largest inner product with it. The items under the 3 user embeddings are shown in Figure 11. It can be observed that different low-dimensional user embeddings have captured different user interests, which enhances the representation ability of URM for the target item set.



Figure 11. Visualization of user embeddings from different [UM] tokens. Each token captures distinct facets of user interest, enabling LLM to jointly express different aspects of the target item set.

Activation of [UM] Tokens on Different Tasks. Figure 12 shows the activation proportion for 8 different [UM] tokens in the CR and SP tasks. A token is activated when the dot product of the user embedding corresponding to that [UM] token

and the target multimodal embedding is used for the final output. We find that different [UM] tokens exhibit variation across different tasks.

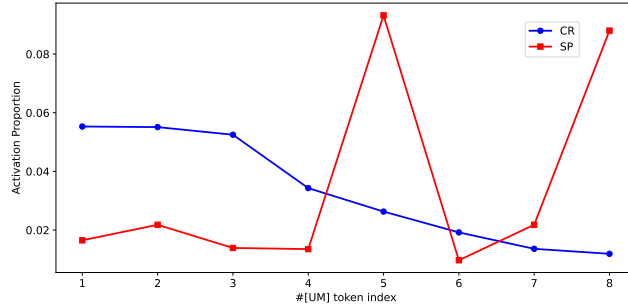


Figure 12. Activation proportion of different [UM] tokens in CR and SP tasks.

The Necessity of Multi-[UM]-Token Format. URM employs multiple [UM] tokens to generate several 128-dimensional output user embeddings, which are subsequently computed alongside the 128-dimensional multimodal item embedding. To explore the effect of the multi-token format, we conducted experiments using higher embedding dimensions and a single token, with the results presented in Table 16. On one hand, when a single token is used to derive a 4096-dimensional user embedding, which is then split into 32 separate 128-dimensional embeddings, the CR recall drops significantly from 0.248 to 0.163. This highlights the advantage of generating multiple user embeddings with different tokens, even when the formal dimension remains constant. On the other hand, increasing the multimodal item embedding dimension from 128 to 4096 (resulting in logits computed in a higher-dimensional space) does not enhance performance compared to the logits derived from multiple tokens, yielding a mere CR recall of 0.203. These findings confirm that the Multi-[UM]-Token design doesn’t solely benefit from its higher dimensionality; rather, it demonstrates superior effectiveness in capturing the diverse interests of users within this specific format.

Table 16. The effect of multi-token (metric: R@1000).

Methods	CR	SP
32 Tokens with 128-d Embeddings	0.248	0.835
1 Token with 4096-d Output User Embedding	0.163	0.774
1 Token with 4096-d Multimodal Item Embedding	0.203	0.781